Cybernetics and Systems Analysis, Vol. 46, No. 4, 2010

SOLVING LINEAR CONSTRAINTS OVER REAL AND RATIONAL FIELDS

V. N. German

UDC 519.852.2

Abstract. *Practical methods are proposed to solve linear constraints over real and rational fields with quantifiers. Secondary problems are observed together with the set of possible solutions in the context of automatic software model verification.*

Keywords: *model verification, linear inequalities, systems of constraints, DNF, quantifier elimination, convex polyhedron.*

INTRODUCTION

Practical program verification has attracted increasing interest in recent years. The ever increasing complexity of software and program projects makes automatic verification a more and more popular and developing field of theoretical and practical studies. Verification can be considered as a tool to struggle against the complexity since it allows automating error detection at early stages of software development thus minimizing the probability of serious errors at late stages of software development. The constantly increasing volume and complexity of modern program projects make it obvious that the struggle against the complexity is a fundamental programming problem. Thus, verification is an important part of the set of methods to solve fundamental programming problems and to reduce the terms of software development. Important components of the verification process are ascertaining whether formulas are true or contradictory in various algebras and finding a set of values on which such formulas are true.

In such verification systems, the original model of a program system is described by specification languages such as MSC, UML, and SDL. Verifying such specifications reduces to proving the formulas obtained from specification transformations. Obviously, to solve and prove formulas, solvers of the formulas in given algebras (or sets of axioms) are necessary. Depending on the situation and on specification description, various solvers can be involved: for first-order formulas of predicates logic [1], Presburger arithmetic [2], for optimization problems, and for linear inequalities and equalities over real numbers. The input formula is analyzed by a verification system and is divided into subformulas with marked type. Then a solver for the given type of formulas is called in interactive mode. The solver outputs the answer whether the formula is true or yields its simplified formula. Thus, the input formula is reduced until a solution is obtained or it is proved that the system has no solution. The present paper considers the set of methods and algorithms to construct one of the solvers, namely, a solver for formulas that consist of systems of linear constraints over the field of real or rational numbers. The input formula for such a solver contains generality and existential quantifiers $\{\exists, \forall\}$, equalities, inequalities, and negation of equality $\{=, \geq, >, \leq, <\}$, logical connectives conjunction, disjunction, and negation $\{\wedge, \vee, \neg\}$. As a result, the solver produces an output formula that contains no quantifiers and can contain free, i.e., not quantified variables. If the input formula is closed, i.e., contains only quantified variables, the solver outputs a logic result: true or false. If the resources are insufficient to find a solution or values of one of the constraints (such as memory used or solution time) are exceeded, the result can be unknown. There are systems that solve specialized problems over a simplified subset of input formulas. In this case, it is justifiable to apply faster special algorithms to efficiently solve a highly specialized problem. The algorithm presented in the paper does not impose constraints on the form of the input formula. The formula should be linear, i.e.,

V. M. Glushkov Institute of Cybernetics, National Academy of Sciences of Ukraine, Kyiv, Ukraine, *vitaly.german@gmail.com*. Translated from Kibernetika i Sistemnyi Analiz, No. 4, pp. 123–133, July–August 2010. Original article submitted April 15, 2010.

 1060-0396/10/4604-0630 ©2010 Springer Science+Business Media, Inc.

a quantified or free variable can be multiplied by a numerical constant only, and the values of unknown variables and constants belong to the class of real or rational numbers. Such an approach results both in a universal solver and the loss of speed and/or impossibility of obtaining a solution for very complex formulas. The complexity of a formula is directly proportional to both the number of constraints (equalities, inequalities, and negations of equality) and the number of bound variables.

Section 1 reviews the available methods and solution algorithms for the problem of constraint resolvability over the field of real numbers. Section 2 presents the framework of the algorithm. Section 3 considers variable elimination algorithms for equality and systems of constraints. Section 4 proposes methods to simplify and remove the redundancy in intermediate formulas.

1. REVIEW OF METHODS

Currently, there are a lot of ways to solve linear inequalities. Depending on approaches and purposes, they can be conventionally grouped as follows:

- algorithms derived from the Fourier–Motzkin algorithm [3];
- simplex method and its derivatives;
- cylindrical algebraic decomposition (CAD) [4];
- fast algorithms for formulas of certain form.

The Fourier–Motzkin algorithm is the first resolving procedure for real number arithmetic. This method was later adapted to solve Presburger arithmetic, for example, Omega Test algorithm [5]. The Fourier–Motzkin algorithm is quite simple and universal. Its shortcoming is the superexponential complexity in the worst case. The simplex method is tried-and-true and shows even polynomial average time. This method is very suitable to test whether a system of linear quantifier-free inequalities, equalities, and negations of equality has a solution. The simplex method is applicable for fast system compatibility test. The incremental simplex method [6] allows efficiently testing the compatibility of a new constraint added in a consistent set of constraints. This property of the incremental simplex method can be applied at the stage of the minimization of intermediate formulas (Sec. 4). The simplex method does not allow simplifying a formula by eliminating quantifiers; therefore, it cannot be used as the basic algorithm to solve a problem posed. The CAD algorithm developed by Collins [4] allows solving quantifier elimination problems for both linear and nonlinear systems. The practical application is limited because of the high computational complexity. The next subclass includes algorithms that work only for formulas of certain form, for example, for formulas that contain only two variables in each set of constraints or, in other words, conjunction of inequalities. Such an approach allows solving a problem in virtually linear time and space [6]. The extreme points method proposed by Lassez [8] is of exponential complexity of variable elimination.

2. FRAMEWORK OF THE ALGORITHM

Inside the system, a formula is equivalently transformed so as to contain only inequalities $\{<, \leq\}$ and the existential quantifier. Such a basis needs the negation of equality to be transformed into the disjunction of inequalities, which may exponentially increase the number of inequalities at the stage of reducing the formula to the disjunctive normal form (DNF). The methods to reduce the size of an intermediate formula during the conversion to the DNF are considered in Sec. 3.4. At the upper level, the solver follows the algorithm below:

(i) transform generality quantifiers into existential quantifiers, see 3.1;
(ii) reduce the function to the DNF;
(iii) eliminate the innermost quantifier;
(iv) simplify;
(v) if there are quantifiers, go to Step (ii);
(iv) output the result.

3. QUANTIFIER ELIMINATION

3.1. General Formula Transformations. A formula can be reduced to the minisphere normal form as follows. The generality quantifier is replaced with the existential quantifier. The existential quantifier is transferred inside the disjunction.

The negation of disjunction is taken out of the brackets and, finally, double negation of the formula is replaced with the formula itself. The formal rules of such a transformation are as follows:

$$\forall v\colon f \to \neg\exists v\colon \neg f,$$

$$\exists v\colon (f_1 \vee \ldots \vee f_n) \to \exists v\colon f_1 \vee \ldots \vee \exists v\colon f_n, \tag{1}$$

$$\neg(f_1 \vee \ldots \vee f_n) \to \neg f_1 \wedge \ldots \wedge \neg f_n, \quad \neg\neg f \to f.$$

3.2. Solving Equalities. We have a system of linear inequalities and equalities. Variables can both appear freely or be bound with the existential quantifier. If the bound variable appears in an equality, it is eliminated by substituting its value into all the other equalities and inequalities. This step repeats until all the equalities and/or bound variables are eliminated. All the equalities and inequalities inside the system are presented as a matrix. One substitution of a variable is equivalent to one step in the Gauss method of solving systems of linear equalities [9]. If a system of equalities contains free variables, it should be simplified and represented in the solved form. This method is described in more detail in Sec. 4.1.

3.3. Solving Inequalities. Let us pass to the method of variable elimination in a system of linear inequalities. It was discovered by Fourier (1826) and can be considered as an expansion of the Gaussian method 3.2 for inequalities. This method was temporarily forgotten and invented anew by Dines and Motzkin [10].

The main idea of the method is rather simple and implies applying the following transformations to the original system of inequalities:

$$a \le bx \wedge cx \le d \leftrightarrow \frac{a}{b} \le x \le \frac{d}{c} \to \frac{a}{b} \le \frac{d}{c}, \tag{2}$$

where x is the variable to be eliminated; a, b, c, and d are numerical constants. The above-mentioned transformations should be applied to all the combinations of the right- and left-hand inequalities. By a right-hand inequality we mean the inequality where the variable to be eliminated or, in other words, the bound variable with positive coefficient is on the right. By a left-hand inequality we mean the inequality where the variable to be eliminated with positive coefficient is on the left. The proposed solver applies equivalent transformations, and inequalities are specified in the matrix form. Let us consider a system of inequalities as a matrix. This system can contain both equalities and inequalities. The type of a certain system is determined by a type vector. The length of such a vector coincides with the number of inequalities and rows in the matrix. Let us consider the Fourier–Motzkin algorithm in the matrix form. Assume that there is a system of inequalities $S[m, n] = Ax \le b$, which contains m linear inequalities and n variables. This system is specified by the matrix of coefficients (A) and the vector of constants (b). For simplicity, we assume that only unstrict inequalities appear in the system and consider the solution for the general case:

$$\begin{bmatrix} a_{11} & a_{12} & \ldots & a_{1n} \\ a_{21} & a_{22} & \ldots & a_{2n} \\ \ldots & \ldots & \ldots & \ldots \\ a_{m1} & a_{m2} & \ldots & a_{mn} \end{bmatrix} \le \begin{bmatrix} b_1 \\ b_2 \\ \ldots \\ bm \end{bmatrix}. \tag{3}$$

To find a solution of the system of inequalities, variables should be excluded from the system. To this end, we take an arbitrary column of the matrix (let it be the first column). The vector of coefficients of such a matrix is $a_1^T = [a_{11}\ a_{21}\ \ldots a_{m1}]$. Both positive and negative coefficients can appear in this vector. If the vector a_1^T contains only positive or only negative coefficients, then such a system always has a solution, irrespective of the variable. Therefore, we delete this column and set all the values of the coefficients of the vector a_1^T to zero. If the system cannot be solved trivially, we form an eliminating matrix $C_1 \ge 0$ such that

$$\frac{C_1 \cdot Ax \le C_1 \cdot b}{A'x' \le b'} \tag{4}$$

after the matrix of coefficients and the vector of constants are multiplied by the eliminating matrix C_k, the resultant matrix A' does not contain a column with the index k. The vector b' contains updated coefficients. This means that the resultant system is equivalent to the original one but does not depend any more on the variable that was at the position k. Let us consider the construction of such a matrix. Take all the possible combinations (product) of positive and negative coefficients.

For example, let $a_{ij}^{+} > 0,\ a_{ij}^{-} < 0,\ i \in [1 \ldots m],\ j \in [1 \ldots n]$:

$$A = \begin{bmatrix} a_{11}^{+} & a_{12} & \ldots & a_{1n} \\ a_{21}^{-} & a_{22} & \ldots & a_{2n} \\ a_{31}^{+} & a_{32} & \ldots & \ldots \\ a_{41}^{-} & a_{42} & \ldots & a_{4n} \end{bmatrix}, \tag{5}$$

$$C_1 = \begin{bmatrix} -a_{21}^{-} & a_{11}^{+} & 0 & 0 \\ -a_{41}^{-} & 0 & 0 & a_{11}^{+} \\ 0 & a_{31}^{+} & -a_{21}^{-} & 0 \\ 0 & 0 & -a_{41}^{-} & a_{31}^{+} \end{bmatrix}. \tag{6}$$

Let the number of positive coefficients in the vector a_1^T be k^+, the number of negative coefficients be k^-, and the number of zero coefficients be k^0. Then the number of rows in the matrix C is

$$m' = k^+ k^- + k^0. \tag{7}$$

The number of columns is equal to the number of columns in the matrix A:

$$C_1 A = \begin{bmatrix} -a_{21}^{-} & a_{11}^{+} & 0 & 0 \\ -a_{41}^{-} & 0 & 0 & a_{11}^{+} \\ 0 & a_{31}^{+} & -a_{21}^{-} & 0 \\ 0 & 0 & -a_{41}^{-} & a_{31}^{+} \end{bmatrix} \begin{bmatrix} a_{11}^{+} & a_{12} & \ldots & a_{1n} \\ a_{21}^{-} & a_{22} & \ldots & a_{2n} \\ a_{31}^{+} & a_{32} & \ldots & \ldots \\ a_{41}^{-} & a_{42} & \ldots & a_{4n} \end{bmatrix}. \tag{8}$$

The resultant system A' has the form

$$A' = \begin{bmatrix} -a_{21}^{-}a_{11}^{+} + a_{11}^{+}a_{21}^{-} & -a_{21}^{-}a_{12} + a_{11}^{+}a_{22} & \ldots & -a_{21}^{-}a_{1n} + a_{11}^{+}a_{2n} \\ -a_{41}^{-}a_{11}^{+} + a_{11}^{+}a_{41}^{-} & -a_{41}^{-}a_{12} + a_{11}^{+}a_{42} & \ldots & -a_{41}^{-}a_{1n} + a_{11}^{+}a_{4n} \\ a_{31}^{+}a_{21}^{-} - a_{21}^{-}a_{31}^{+} & a_{31}^{+}a_{22} - a_{21}^{-}a_{32} & \ldots & \ldots \\ a_{41}^{-}a_{31}^{+} + a_{31}^{+}a_{41}^{-} & a_{41}^{-}a_{32} + a_{31}^{+}a_{42} & \ldots & a_{41}^{-}a_{32} + a_{31}^{+}a_{42} \end{bmatrix}. \tag{9}$$

After simplification, the first column of the matrix A' obviously consists of zero elements only. Thus, the resultant system does not depend any more on the variable that specifies the first column:

$$A' = \begin{bmatrix} 0 & a'_{12} & \ldots & a'_{1n} \\ 0 & a'_{22} & \ldots & a'_{2n} \\ 0 & a'_{32} & \ldots & a'_{3n} \\ 0 & a'_{42} & \ldots & a'_{4n} \end{bmatrix}, \quad b' = C_1 b = \begin{bmatrix} b'_1 \\ b'_2 \\ b'_3 \\ b'_4 \end{bmatrix}. \tag{10}$$

In the example, there are two positive and two negative coefficients and no zero coefficients. As a result, the numbers of inequalities in the resultant and original matrix are identical. In particular, some specialized solvers use the property that constraints in systems with two variables and two inequalities do not grow. Let us calculate the number of resultant inequalities for an arbitrary number of inequalities. Let the number of inequalities be *m*, the numbers of positive and negative coefficients be identical, and independent constraints be absent. Then the number of rows of the resultant matrix after the elimination of one variable is equal to

$$\left(\frac{m}{2}\right)^2. \tag{11}$$

For *n* bound variables, the dimension of the resultant formula can be

$$\left(\frac{m}{2}\right)^{2^n}. \tag{12}$$

Thus, the algorithm is of superexponential complexity, at worst, for an arbitrary number of constraints and variables. The practical realization can be improved because the resultant set of inequalities is often redundant after the elimination of a variable. Thus, applying an efficient algorithm of finding and removing redundant inequalities allows using the Fourier–Motzkin algorithm to solve real problems. One of such algorithms is considered in Sec. 4.2.

In the beginning of the section, we assumed that the system contains only unstrict inequalities. In the general case, there can also be equalities and unstrict inequalities in the system. The type of a constraint is determined by an additional type vector whose dimension coincides with the number of rows in the matrix A, i.e., by the number of constraints in the system. All the equalities will be transferred to the resultant system since the coefficient of the variable being eliminated is always zero for an equality. This statement is true since the equalities with nonzero coefficients have already been eliminated at the previous step (see Sec. 3.2). Whether the resultant inequality is strict or unstrict is determined in forming the eliminating matrix C. If two chosen inequalities are of identical types, the resultant inequality will be of the same type; otherwise, the resultant inequality will be strict.

The paper [11] proposes an alternative for the Fourier–Motzkin elimination. A variable in a system of linear inequalities is excluded by calculating extreme points. This method excludes all the variables simultaneously by using a quasidual representation of the system S of linear constraints in view of the variables to be excluded. The method follows from the generalized linear optimization problem with the vector-valued objective function Φ over a bounded polyhedron. Then the extreme points of the polyhedron $\Phi(P)$ determine the final system of linear inequalities, which describe the projection of solutions of the set S into the space of the other variables. One more efficient alternative is proposed in [12].

3.4. Negations of Equalities. Negations of equalities play an important role in solving linear inequalities since the computational complexity of the (in any case complex, see Sec. 3.3) quantifier elimination algorithm for real numbers depends on how negations of equalities will be solved. The naive realization is excluding inequalities from the system by representing negations of equality (we will also call them negative constraints) in terms of strict inequalities:

$$x \neq b \leftrightarrow b < x \vee x < b. \tag{13}$$

Thus, the resultant formula no longer contains inequalities. A shortcoming of such an approach is its complexity. Assume that the original system has the form

$$x_1 \neq b_1 \wedge x_2 \neq b_2 \wedge \ldots \wedge x_n \neq b_n \wedge f, \tag{14}$$

where f is an arbitrary linear formula not containing the negation of equality. Then applying the rule once and removing the disjunction yield

$$(x_1 < b_1 \wedge x_2 \neq b_2 \wedge \ldots \wedge x_n \neq b_n \wedge f) \vee (x_1 > b_1 \wedge x_2 \neq b_2 \wedge \ldots \wedge x_n \neq b_n \wedge f). \tag{15}$$

Each conjunctive of disjunction will generate two more inequalities. Thus, after replacing all the negations of equality with inequalities, the resultant number of inequalities is

$$2^n. \tag{16}$$

Even for the original, totally for ten ($n = 10$) negations of equality, the resultant system will contain 1024 inequalities. There is a better way for real and rational numbers. The system of generalized linear constraints [13] is a finite conjunction of positive and negative linear constraints. A key to process generalized linear constraints is the mutual independence of negative linear constraints. The system has solutions if and only if the positive constraints f together with one of the negative constraints $x_i \neq b_i \wedge f$, $i \in [1 \ldots n]$, have solutions. Geometrically, this means that the polyhedron belongs to a finite set of affine sets if and only if it belongs to one of them. Lassez and McAloon showed in [13] that the Fourier–Motzkin method can easily be adapted to calculate the affine hull of a polyhedron. Passing to the system of generalized linear constraints needs a modification of the variable elimination algorithm. Two variants of the algorithm are presented in [14].

4. REMOVING THE REDUNDANCY

4.1. Simplifying a System of Equalities. The conjunction of linear equalities $Ex = f$ is a part of the system of constraints. In the original formula or in the formula resulted from the elimination of a variable, the system of linear equalities can be redundant and/or be in unsolved form. Such a representation is redundant since an infinite number of equalities can specify one solution. Therefore, the system of equalities should be represented in the solved form. A system of linear equalities is known to be inconsistent if and only if it can be represented in the solved form. Classical algorithms of

linear algebra can be applied to solve a system of equalities [9]. From the practical point of view, there is a lot of libraries to solve a system of linear equalities, for example, Boost Libraries, which is almost standard for C ++ [15].

During the realization, there may appear a situation where one system of equalities contains several independent systems of equalities while a standard function of the solution of linear equalities expects only one system of equalities at the input, without redundant inequalities. To solve this problem, the following is necessary:

- decompose the system $Ex = f$ into independent systems of equalities $Ex = f \to \{ E_1 x = C_1, \ldots, E_n x = f_n \}$;
- normalize and remove the redundancy of each of the independent systems, $Simplify\,(Normalize\,(E_i x = f_i))$, $i \in [1 \ldots n]$;
- unite all the solutions into one system $SolvedForm\,(Ex = f)$.

It is assumed that $SolvedForm\,(Ex = f)$ returns the False value if the system cannot be solved. A remarkable property of such a system is that its solution can be obtained by substituting a specific value instead of one of the variables.

4.2. Simplifying a System of Inequalities and Equalities. The second part of the system of linear constraints is a system of linear inequalities and equalities $\{Ax < B_1,\ Ax \le B_2,\ Ex = f\}$. The simplification of the system of inequalities allows expanding the domain of practical application of the solver of linear inequalities. The simplification of inequalities cannot be considered without equalities since such simplifications can result in new equalities. Since the number of inequalities can promptly increase at each step of variable elimination (see Sec. 3.3), the struggle against redundancy is an integral problem to be solved efficiently. The system of constraints should be normalized. This means that all the inequalities that contain identical variables should have identical absolute values of the coefficients with respect to one of the variables. Such a normalization guarantees that the coefficients of parallel constraints will be equal or have opposite signs. This statement obviously propagates only to integer and rational coefficients. The idea at the bottom of the technique is parallel half-spaces (hyperplanes) that specify the original system:

$$a_1 x_1 + f_1 < b_1 \wedge a_1 x_1 + f_1 < b_2 \to a_1 x_1 + f_1 < \min(b_1,\ b_2) \text{ if } \quad a_1 > 0, \tag{17}$$

$$a_1 x_1 + f_1 < b_1 \wedge a_1 x_1 + f_1 < b_2 \to a_1 x_1 + f_1 < \max(b_1,\ b_2) \text{ if } \quad a_1 < 0.$$

Let us choose two inequalities that differ only in constant. Depending on the coefficient of the chosen variable, let us replace these two inequalities with one whose constant is minimum (if the coefficient is positive) or maximum (otherwise). The example considered above suppose that we deal only with strict inequalities. However, the situation is complicated if we consider pairwise combinations of strict and unstrict inequalities and equalities with each other. In this case, it is necessary to additionally consider combinations of the types of inequalities T_1, T_2 or, in other words, transitive closure of constraints. In this case, the type of the resultant inequality can be determined from the table below.

T_1	T_2		
	$a = b$	$a \le b$	$a < b$
$=$	$=$	$=$	False
$\le$	$=$	$\le$	$<$
$<$	False	$<$	$<$

Some combinations cause contradictions in the original system. In the table, this is denoted as False. In this case, $RemRedundant\,(\{Ax < B_1,\ Ax \le B_2,\ Ex = f\})$ returns False. After all the parallel half-spaces and hyperplanes that have equal coefficients of the variables are simplified, it is possible to start simplifying the parallel constraints that differ in the sign of the coefficients:

$$a_1 x_1 + f_1 \le b_1 \wedge -a_1 x_1 - f_1 \le b_1 \to a_1 x_1 + f_1 = b_1. \tag{18}$$

If the inequalities with opposite coefficients of the chosen variable contain different constants b, then we test whether these two constraints specify a convex polyhedron. Nothing can be simplified in this case. If not, we have a contradiction. If the constants are also equal, the resultant constraint can be found from the table below.

T_1	T_2		
	$a = b$	$a \le b$	$a < b$
$=$	$=$	$\le$	False
$\ge$	$\ge$	$=$	False
$>$	False	False	False

In the table above, the inequalities $\{>,\geq\}$ are only for descriptive reasons; we mean that we deal with the negative coefficient of the chosen variable and inequalities $\{<,\leq\}$. At this stage, a pair of inequalities can be replaced with Eq. (18) or a contradiction can be found. In the realization, it is important to quickly find parallel constraints with both identical and opposite coefficients. One of the possible solutions is to apply hashing so that, given inequality and system of constraints, the search function returns an iterator on the list of hyperplanes parallel to the constraint specified.

4.3. Operation *gist*. The simplifications of conjunction or system of linear constraints were considered earlier; however, this technique is not sufficient to obtain the results and intermediate calculations without redundancy. The gist operation was proposed in [16] and can be used, as will be shown below, to simplify the disjunction of systems of linear constraints.

Definition. *gis p given q* is the conjunction that contains the minimum subset of constraints from p such that $((gist\ p\ given\ q) \wedge q) = (p \wedge q)$.

Intuitively, *gis p given q* is the new information contained in p provided that *q* is known. By definition, *gist* always contains no more constraints than the original system of constraints does.

Suppose that the system of constraints *q* is solvable, then the algorithm to calculate gist p given q := is as follows:

1) if (p is *true*) than
2) return *true*
3) else
4) let c be constraint in p
5) if $(p^{c}_{\sim(c)}) \wedge q$ is *consistent* then
6) return $(c \wedge$ gist p^{c}_{True} *given* $(q \wedge c))$
7) else
8) return (*gist* p^{c}_{True} *given* q)

Here, p^{oldc}_{newc} is p, where the element *oldc* is replaced with the element *newc*. Such an algorithm is laborious since it contains repeated consistency tests for systems of linear constraints, each being rather toilful. The problem can be solved more efficiently by testing special cases.

- For each constraint e in the set of constraints p check whether *e* follows from any constraint in p or q. If so, then *e* is redundant and will not appear in *gist*.
- Check whether some of the variables has upper constraints in p but does not have in q. If so, then at least one of the upper constraints should appear in *gist*. A similar check should be done for lower constraints.
- If there is no constraint e' in p or q such that the scalar product of the normals *e* and e' is positive, then *e* should be in *gist*.

If the constraint is determined to be redundant, it no longer participates in the further conclusions about the redundancy of other inequalities. Such a procedure of fast test of special cases often calculates *gist* completely or considerably simplifies the problem before calling the general algorithm.

4.4. Simplifying the Disjunction of Constraints. The operation *gist* defined in Sec. 4.3 can be used to check tautologies. If $(gist\ p\ given\ q) == True$, then $p \Rightarrow q$ is a tautology.

Let we have a formula $A \vee B$, where A and B are arbitrary systems of constraints. Let us use the operation *gist* to test the validity of $A \Rightarrow B$. If so, then $A \vee B \xrightarrow{A \Rightarrow B} A$.

Using such method can simplify the disjunctions of an arbitrary number of systems of constraints. To check for tautology, the function *gist* is used, which stops once it becomes obvious that the result of the operation will not be equal to True.

5. REDUCING TO DNF

For the quantifier elimination algorithm for each of the conjucts to start, a formula should be reduced to the normal form. A shortcoming of the "naive" algorithm of reducing to DNF is the fact that the number of literals can exponentially increase during the operation of the algorithm. To struggle against the increase in the size of both the intermediate and resultant formula in the solver, it is necessary to reduce to DNF considering that the validity of the final formula can be proved. In the case of systems of linear constraints, it is possible to prove uniquely whether the conjunction of constraints specifies a convex polyhedron or an empty set. The following tests are carried out in calculating the DNF:

- if a new constraint is introduced in the system of constraints, the new system is tested for consistency; in case of a contradiction, the entire system is deleted;
- redundancy test and simplification if two different conjunctions of constraints are united into one convex polyhedron.

More complex heuristics are also implemented with the use of the *gist* operation:

- removal of repeated disjuncts and tautologies (see Sec. 4.4);
- the minimum representation of formulas $A \wedge \neg B$ with the use of *gist*.

By definition, $A \wedge (gist\ B\ given\ A) \equiv (A \wedge B)$. Then if a formula contains the conjunction of a system of constraints and the negation of another system of constraints, then to reduce the number of constraints in the negation, the operation *gist* can be used when expanding the negation:

$$A \wedge \neg B \equiv A \wedge \neg(A \wedge B),$$

$$A \wedge \neg(A \wedge B) \equiv A \wedge \neg(A \wedge (gist\ B\ given\ A)),$$

$$A \wedge \neg(A \wedge (gist\ B\ given\ A)) \equiv A \wedge \neg(gist\ B\ given\ A).$$

The resultant system $A \wedge \neg(gist\ B\ given\ A)$ often has a smaller number of constraints than the system $A \wedge \neg B$ has. This it is obvious since by definition, *gist B given A* always has an equal or smaller number of expressions than *B*.

Along with the methods presented above, the formulas can be further reduced by algorithms that have a guaranteed lower complexity than the naive algorithm has. One of the first developments is the subexponential algorithm described in [17]. The later studies [18, 19] propose various improvements, which accelerates finding the DNF.

6. PRACTICAL RESULTS

The majority of the presented methods are implemented in a solver of linear constraints for real and rational numbers. This solver is built-in to the APS algebraic programming system [20]. There exists also a Linear Solver version independent of the APS and available for free loading [21]. The APS and the solver were used to successfully verify two different models of time-triggered protocol (TTP) [22] (for more detail see [23]). A three-node model is presented in [24] and consists of 21 variables and 33 constraints. The solver solves such a formula on a modern personal computer in several minutes. The solution time depends on the values of the coefficients tau1, tau2, tau3 $\in$ (0 ... 1] and varies from several seconds to several minutes.

Let us present an example of a two-node model:

```
Resolve[ForAll [{d, p, c, newc, newp},
(
    !((newc == – D*(c + d) – p) &&
    (newp == p + c + d) &&
    (– 1 ≤ d) && (d ≤ 1) && (– M ≤ p) && (p ≤ M) &&
    (1 – M ≤ p + c) && (p + c ≤ M –1) &&
    (– M ≤ c) && (c ≤ M)
) || (
    (1 – M ≤ newp + newc) && (newp + newc ≤ M –1)
)
) && (M > 1) && (D > 0) && (D <= 1)
], {M, D}, Reals]
```

Out = (M > 1 && 2/(1 + M) ≤ D ≤ 1)

It is required to find how the free variable *M* depends on the parameter *D* for which the formula above is true. The system is written in Wolfram Mathematica 5.2. Mathematica uses the CAD algorithm to eliminate quantifiers and allows eliminating quantifiers also for problems of higher order. Both Mathematica and Linear Solver have successfully coped with a two-node model, Mathematica allows using the coefficient *D* in symbolic form. A tree-node model [24] is a challenge for Mathematica 5.2 on a 2 Gb RAM computer. After several minutes, the calculations are terminated with an insufficient-memory message. At the same time, the Linear Solver solves a three-node model, as indicated above.

The current version of the Linear Solver does not support symbolic coefficients, which may be implemented in future versions. Testing the performance compared with that of the available solvers can also be an interesting problem for the forthcoming studies.

CONCLUSIONS

Creating a good solver for natural and real numbers is a challenge; no unique efficient algorithm is available to solve it. The general quantifier elimination algorithm together with the naive algorithm of reducing to DNF is, at worst, of superexponential complexity, which makes it suitable to solve only a limited scope of problems. Nontrivial algorithms often need a great many of sources, studies, and experiments to be searched and processed. The present paper has considered a set of methods underlying the solver for natural and real numbers, applicable to solve many practical verification problems. The realization of the solver allows obtaining practical results for real applications, though it has limitations that should be removed to expand its application; references to modern methods, alternatives, and ways of further studies have also been presented.

REFERENCES

1. http://en.wikipedia.org/wiki/First-order_logic.
2. http://coq.inria.fr/V8.1pl4/refman/Reference-Manual022.html.
3. http://en.wikipedia.org/wiki/Fourier%E2%80%93Motzkin_elimination
4. G. E. Collins, "Quantifier elimination by cylindrical algebraic decomposition — twenty years of progress," in: B. F. Caviness and J. R. Johnson (eds.), Quantifier Elimination and Cylindrical Algebraic Decomposition, Springer–Verlag, New York (1998), pp. 8–23.
5. http://www.umiacs.umd.edu/~hismail/Omega/node3.html.
6. G. Badros, A. Borning, and P. Stuckey, "The Cassowary linear arithmetic constraint solving algorithm," ACM Trans. on Computer-Human Interaction (TOCHI), **8(4)**, 267–306 (2001).
7. http://research.microsoft.com/en-us/um/redmond/projects/z3/index.html.
8. J.-L. Lassez, "Querying constraints," in: Proc. ACM Conf. on Principles of Database Systems (ACM 089791-352-3/90/0004/0288), Nashville (1990).
9. http://en.wikipedia.org/wiki/Gaussian_elimination
10. B. C. Eaves and U. G. Rothblum, "Dines–Fourier–Motzkin quantifier elimination and an application of corresponding transfer principles over ordered fields," Math. Program., **53**, No. 3, 307–321 (1992).
11. T. Huynh, L. Joskowicz, C. Lassez, and J. Lassez, "Practical tools for reasoning about linear constraints," Fundam. Inf., **3–4**, 357–380 (1991).
12. C. Lassez and J.-L. Lassez, Quantifier Elimination for Conjunctions of Linear Constraint via a Convex Hull Algorithm: Techn. Rep., IBM T. J. Watson / Research Center, Yorktown Heights (1991).
13. J. L. Lassez and K. McAloon, "A canonical form of generalized linear constraints," J. Symbolic Comput., **33**, 147–160 (1992).
14. J.-L. Imbert, "Variable elimination for disequations in generalised liner constraint systems," The Computer J., **36**, No. 5 (1993).
15. Boost Libraries, www.boost.org
16. W. Pugh and D. Wonnacott, "Going beyond integer programming with the omega test to eliminate false data dependences," IEEE Trans. on Parallel and Distributed Systems, **6**, No. 2, 204–211 (1995).
17. N. Bshouty, "A subexponential exact learning algorithm for DNF using equivalence queries," Information Processing Letters, **59**, 37–39 (1996).
18. J. Tarui and T. Tsukiji, "Learning DNF by approximating inclusion-exclusion formulae," in: Proc. IEEE Conf. on Comput. Complexity (1999), pp. 215–220.
19. A. R. Klivans and R. A. Servedio, "Learning DNF in time $2^{O(n^{1/3})}$," J. Comput. Syst. Sci., **68(2)**, 303–318 (2004).
20. http://www.springerlink.com/content/htq68m20q121t3jw/.
21. http://code.google.com/p/constraints-solver-over-r/.
22. http://en.wikipedia.org/wiki/Time-Triggered_Protocol.
23. A. B. Godlevsky and S. N. Svirgunenko, "Using linear models in optimizing parameters of time-triggered protocols," Cybern. Syst. Analysis, **42**, No. 3, 328–334 (2006).
24. http://www.risc.uni-linz.ac.at/projects/intas/Timisoara/Presentations/German/German.pdf.